

03 | תיאור המערכת

לאחר שיש בידנו כבר אלגוריתמים בסיסיים לתנועה בליסטית וזיהוי מקום וזמן הנחיתה נגדיר את המערכת כולה

State Variables

עבור כל איום i (טיל) בזמן נתון t נגדיר את הפרמטרים הבאים:

Impact Damage הפוטנציאלי (V_i) :

- הוא מחושב כמכפלה של גודל האיום ברגישות אזור הפגיעה. $V_i = W_i \cdot Z_i$
 - כאשר W_i הוא גודל ראש הנפץ (סוג הטיל): רקטה קלה = 1, טיל בינוני = 3 וטיל כבד = 5.
 - כאשר Z_i הוא משקל איזור הפגיעה - ים = 0, שטח פתוח = 1, ישוב כפרי = 3, עיר = 9.
- דוגמה 1:** איום רקטה קלה לעיר $9 < 5$ איום טיל כבד לשטח פתוח.
- דוגמה 2:** טיל בינוני לישוב כפרי $9 = 9$ רקטה קלה לעיר - במקרה כזה נתחשב בעוד משתנים (מפורט למטה)
- הערה: מוסבר למטה איך להעריך את איזור הפגיעה.

Time To Impact - $TTI_i(t)$ - זמן שנותר עד לפגיעת האיום ה- i בקרקע, מתעדכן בכל פעימת שעון שזה זמן t .

- הוא מוערך פעם אחת כאשר המטרה בטווח היירוט.
- רקטות קלות (קצרות טווח) עפות במסלול בליסטי נמוך ואיטי יחסית לכן ה- TTI שלה בעת הגילוי יהיה יחסית גבוה
- טילים כבדים בליסטיים ארוכי טווח ומגיעים לגבהים גדולים ונופלים במהירות גבוהה לכן בעת גילוי TTI יהיה יחסית נמוך
- את ערכי ה- TTI המיטביים למערכת אבדוק אחר כך.

Remaining Dwell Time - $D_{rem,i}(t)$: זמן הלייזר הרציף שעוד נותר כדי להשמיד את המטרה.

- נשמור על זמנים נמוכים יחסית למען הסימולציה: רקטה קלה: 0.5 שניות, רקטה בינונית 1.5 שניות וטיל כבד 2.5 שניות.
- אם הטיל רק התגלה, ערך זה שווה ל- Dwell Time המקורי שלו.
- לדוגמה:** עבור $dwell\ time = 3$, אם המיירט כבר "מבשל" אותו במשך שנייה - הערך ירד ל-2 שניות.
- דוגמת העמסת מערכת:** עבור מיירט בודד בתרחיש של 10 טילים כבדים: יקח לו 50 שניות לנקות את הגל.

קעת שיש לנו את כל המשתנים נוכל להגדיר את פונקציית הציון (Score Function)

על מנת שהאלגוריתם שלנו יבדל מהנאיבי (שמסתכל רק על TTI), נגדיר פונקציית עדיפות כך שתעדיף מטרות שגורמות נזק רב, קרובות לפגיעה ודורשות מעט זמן השמדה (כדי לפנות את המיירט מהר למטרה הבאה).

$$S_i(t) = \frac{V_i}{(TTI_i(t) + \epsilon) \cdot (D_{rem,i}(t) + \epsilon)} \quad \begin{aligned} TTI_i(t) &\geq D_{rem,i}(t) + T_{safety} \\ TTI_i(t) &< D_{rem,i}(t) + T_{safety} \end{aligned}$$
$$S_i(t) = 0$$
$$T_{safety} = 0.5$$

הסבר מתמטי:

- ככל שהנזק גדול יותר - הציון עולה בקשר ישר.
- ככל שהזמן לפגיעה TTI קטן יותר, המכנה קטן והציון עולה.

- צריך להתחשב במקרה של $TTI + \epsilon < D_{rem,i}(t)$: Lost cause משמע שאין לנו זמן ליירט את הטיל לפני שצמוד מדי לקרקע.
- ככל ש- $D_{rem,i}(t)$ קטן יותר - הציון עולה. נובע מעקרון העלות השקועה, אם כבר השקענו זמן בטיל, כדאי לסיים להשמידו במקום לעבור למטרה עם dwell time מלא.
- הוספנו אפסילון לערכים כדי להימנע מקריסה של division by zero ולהבטיח יציבות נומרית.

דוגמאות להערכת ניקוד :

נקח את הדוגמה מלמעלה ונשבור שיוויון : נגדיר מטרה S_1 להיות טיל בינוני לישוב כפרי ומטרה S_2 להיות טיל קל לעיר.

$$\text{בעת הזיהוי : נניח כי TTI שלהם זהה} = 10 \text{ שניות אז } S_1(t) = \frac{9}{10 \cdot 1.5} = 0.6$$

$$S_2(t) = \frac{9}{10 \cdot 0.5} = 1.8 \text{ אז יש העדפה למטרה 2.}$$

- וזה הגיוני כי טיל "קל" לוקח חצי שנייה לירוט וגם הוא מכון למקום בעל חשיבות עליונה. לכן לא נרצה לנעול את המערכת ל-1.5 שניות של יירוט טיל בינוני כי בזמן הזה טילים קלים יכולים לפגוע.
- בפועל ה-TTI לא יהיה זהה כי רקטות קלות (קצרות טווח) עפות במסלול בליסטי נמוך ואיטי יחסית לכבדים.

מנגנון Target Switching-ו Re-evaluation

כאשר מזהים גל חדש (Saturation) נצטרך לעדכן את מערך המטרות ב-Real Time, אנחנו ממיינים מחדש את המטרות לפי ציון $S_i(t)$ שלהן בסיבוכיות של $O(N \log N)$.
אך יש כאן מלכודת של Thrashing : אם טיל חדש מגיע והציון שלו גבוה ב-0.01 מטיל שנמצא כבר ביירוט, המערכת עלולה להחליף מטרות הלוך ושוב כי היא תחליף מטרות כל הזמן, לכן נגדיר Switching Penalty קטן - מיירט שמטפל במטרה A יעבור למטרה B רק אם מתקיים : $S_B(t) > S_A(t) + \delta$ כאשר δ הוא סף קבוע המייצג "קנס המעבר" (במונחים מציאותיים זה למשל זמן כיוול הלייזר)

שידוך בין K מיירטים ל-N מטרות

ניגש בגישה יעילה ושממתימה למערכת בזמן אמת בגישה חמדנית בשילוב עם חסם ריצוד (Hysteresis) סיבוכיות הריצה תהיה $O(N \log N)$ שמוגבלת ע"י מיון המטרות.

קלט :

1. רשימת N מטרות אקטיביות במערכת לאחר סינון.
 2. רשימת K מיירטים, מצבם (IDLE, ENGAGING) ופרנס למטרתו הנוכחית אם ישנה.
 3. פרמטר δ שמתאר switching penalty (קנס על החלפת מטרה - מייצג זמן שלוקח ללייזר להינעל על מטרה חדשה)
- פלט : זיווג של מיירטים למטרות.

שלב 1 : עדכון סטטוס המיירטים העובדים (ENGAGING)

- לפני שמקצים מטרות חדשות, חייבים לבדוק מה מצב המיירטים העובדים.
- אז עבור כל מיירט שמצבו ENGAGING ופועל על מטרה $M_{current}$:
- האם המטרה הושדמה : כלומר האם $D_{rem} = 0$? אם כן נמחק את המטרה ונגדיר את מצבו להיות IDLE.

שלב 2 : ניקוד ומיון מטרות פנויות (Scoring & Sorting)

1. חשב את הציון $S_i(t)$ לכל המטרות במערכת.

2. מיין את רשימת המטרות בסדר יורד. (מהציון הגבוה לנמוך)
3. סנן מתוך הרשימה את המטרות שכבר מיורטות ע"י מיירטים פעילים. (כעת יש לנו רשימה ממוינת של מטרות חופשיות)

שלב 3 : הקצאת המיירטים הפנויים :

עבור כל מיירט במצב IDLE - עד שיגמרו המיירטים הפנויים או המטרות הפנויות :

1. שלוף את המטרה הראשונה מהרשימה הממוינת (המטרה החופשית עם הציון הגבוה ביותר)
 2. הקצה את המיירט אליה ← המיירט הופך ל-ENGAGING והמטרה מסומנת כ- Targeted.
- שלב 4 : בדיקת כדאיות החלפת מטרה (Target Switching) נועד לטפל בגל חדש ומסוכן של טילים.
- שלב זה יתרחש אם נשארו מטרות חופשיות ברשימה - כלומר יותר טילים ממיירטים בזמן הנוכחי.

עבור כל מיירט במצב ENGAGING שיורה על מטרה $M_{current}$ עם ציון $S_{current}$:

1. קח את המטרה החופשית המסוכנת ביותר שנותרה ברשימה M_{new} עם ציון S_{new} :
2. תנאי החלפה : אם מתקיים $S_{new} > S_{current} + \delta$, אז המיירט עוזב את המטרה הנוכחית ועובר למטרה החדשה והמטרה הישנה חוזרת לרשימת המטרות החופשיות.

סיכום המודל הסופי

בעת זיהוי גל נוסף נחשב : (גל יכול להיות גם טיל בודד !)

שלב 1 : סינון (Filtering)

- עבור כל טיל i , אם $TTI_i(t) < D_{rem,i}(t) + T_{safety}$ נסיר אותו מרשימת המטרות. (הוא מקרה אבוד הטיל יפגע לא משנה מה)

שלב 2 : חישוב ניקוד (Scoring) נחשב עבור רשימת המטרות ציון

$$S_i(t) = \frac{V_i}{(TTI_i(t) + \epsilon) \cdot (D_{rem,i}(t) + \epsilon)}$$

שלב 3 : נבצע שידוכים מיטביים של מיירטים למטרות.

מקרא משתנים :

פרמטר	משמעות	ערכים אפשריים
W_i	סוג איום (Warhead)	קלה (1), בינונית (3), כבדה (5)
Z_i	סיווג אזור פגיעה (Zone)	ים (0), פתוח (1), ישוב כפרי (3), עיר (9)
D_{rem}	זמן שהייה נדרש (Dwell)	קלה (0.5s), בינונית (1.5s), כבדה (2.5s)

מציאת מקום הפגיעה

בעת זיהוי הטיל - נחשב פעם אחת בלבד את יעד הפגיעה המשוער בעזרת האלגוריתם [הזה](#) , ולאחר מכן נעשה אבסטרקציה - נעבור על מערך "ערים" ונבדוק עבור כל נקודת פגיעה חזויה אם מרחק ווקטורי קטן

- חישוב של $O(C)$ כאשר C מספר הערים שנגדיר שהוא יהיה לינארי.

צירפתי פה שוב את האלגוריתם של חישוב מיקום וזמן הנחיתה של המטרה אבל עדיף לקרוא אותו בקבצי ה-

: github

```
import * as THREE from 'three';

/**
 * Predicts impact position and TTI (time to impact) given detection
 * position and velocity.
 * NOTE : it's 3D space so we find the rotation axis using cross
 * product and rotate the position vector.
 */
export interface ImpactPrediction {
  impactPosition: THREE.Vector3;
  timeToImpact: number; // in seconds
}

export function predictMissileImpact(
  detectedPos: THREE.Vector3,
  detectedVel: THREE.Vector3,
  gravity: number = 2,
  planetRadius: number = 1
): ImpactPrediction | null {

  // 1. some environmental context
  const distFromCenter = detectedPos.length();
  const currentHeight = distFromCenter - planetRadius;
  // If it's already on the ground (or below), return current
  position, nothing to work with.
  if (currentHeight <= 0.001) return { impactPosition:
    detectedPos.clone(), timeToImpact: 0 };

  // 2. Find the 3D velocity - x,y into x,y,z using "up direction"

  // The "Up" direction is simply the direction from center to
  current position
  const upDir = detectedPos.clone().normalize();
  // 2.1 FIND v_y (Vertical Speed)
  // We project the Velocity vector onto the Up vector (Dot
  Product).
  // If result is positive, it's flying up. If negative, it's
  falling.
  const current_v_y = detectedVel.dot(upDir);
```

```

    // 2.2 FIND v_x (Horizontal Speed)
    // We remove the vertical component from the total velocity.
    What's left is horizontal.
    // Vector_Horizontal = Velocity - (v_y * UpDir)
    const velocityVerticalVector =
upDir.clone().multiplyScalar(current_v_y);
    const velocityHorizontalVector =
detectedVel.clone().sub(velocityVerticalVector);
    const current_v_x = velocityHorizontalVector.length();

    // 3. Solve the Time to impact (Quadratic Formula) ---
    // We want to know when Height = 0. (landing = height of 0)
    // Equation: 0 = CurrentHeight + (v_y * t) - (0.5 * g * t^2)
    const a = 0.5 * gravity;
    const b = -current_v_y;      // Note the sign flip for standard
quadratic form
    const c = -currentHeight;    // Note the sign flip

    const discriminant = (b * b) - (4 * a * c);

    // if the discriminant is negative, the object will never land
(safety check)
    if (discriminant < 0) {
        console.warn("Object will never land (escaping orbit?)");
        return null;
    }

    // Solves for t. We want the larger result (the future impact).
    const t = (-b + Math.sqrt(discriminant)) / (2 * a);

    // 4. Calculate Impact Location

    // We know how long it flies (t) and how fast it moves sideways
(v_x).
    // Arc Length = Speed * Time
    const distanceToTravel = current_v_x * t;

    // Convert distance to Angle (in radians)
    // Angle = ArcLength / Radius
    const angleToTravel = distanceToTravel / planetRadius;

    // We need an Axis to rotate around.

```

```

// The axis is perpendicular to both Position and Velocity (Cross Product).
const rotationAxis = new THREE.Vector3()
    .crossVectors(detectedPos, detectedVel)
    .normalize();

// Create the final position by rotating the CURRENT position
const finalPos = detectedPos.clone()
    .applyAxisAngle(rotationAxis, angleToTravel) // Rotate along
the path
    .setLength(planetRadius); // Snap to surface

return { impactPosition: finalPos, timeToImpact: t };
}

```

פערי מציאות

1. נתעלם מאילוצי מיקום וטווח וחלוקה לאשכולות אז נניח כי כל המיירטים יכולים לטפל בכל המטרות למען פשטות.

- אם היינו צריכים להתחשב בטווחים של מיירטים אז היינו צריכים אלגוריתמים כבדים יותר של שידוך וזה ישפיע על ביצועי הסימולציה שארצה להציג את התוצאות יותר מדי.

2. מה עם הקצאת 2 מיירטים למטרה אחת? מבחינה פיזיקלית לייזר עובד על הצטברות אנרגיה תרמית אז אם שני לייזרים יורים על אותה המטרה, כמות האנרגיה תצטבר ו-dwell time יתקצר אבל זה יסבך יותר מדי את האלגוריתם וגם נוכל אלגוריתמית "לכוון את כל המיירטים לאותה המטרה לייזר מיידי", נוכל לתרץ גם ע"י thermal blooming : כדי להשמיד טיל, הלייזר צריך לפגוע בנקודת תורפה ספציפית וממוקדת אז אם שני קרני לייזר ממקורות שונים מנסים לפגוע באותו סנטימטר מרובע - האוויר באותו איזור יתחם יותר מדי.

3. פערים נוספים : dwell time לא קבוע - למשל תנאי מזג אוויר מגדילים אותו, ה-TTI קשה לחיזוי כי יש האצה פיזיקלית ע"י מנועים.

לסיכום : יש יותר מדי משתנים להתחשב בהם כדי שהמערכת תהיה מציאותית אז נגדיר את חלקם - כאלו שלא יגבילו אותנו אלגוריתמית ומספיקים כדי לתאר תרחיש יחסית מציאותי, אחרת צריך גישות אחרות הרבה יותר מורכבות אלגוריתמית.